

C++20 Modules in a Qt Desktop Application

A Post-Migration Guide

Sourcetrail-TS · macOS · Homebrew LLVM/Clang 22.1.8 · CMake ≥ 4.4 · Ninja · Qt 6
July 2026

Abstract. This document conserves the findings of migrating Sourcetrail-TS — a Qt 6 desktop application with an AUTOMOC'd GUI, a Clang-libTooling indexer, and stdexec-based concurrency — to C++20 named modules as a flag-gated **dual build**: the same headers keep building classically while a parallel module build (`-DSOURCETRAIL_CXX_MODULES=ON`) compiles them into ~15 named modules with 44 module units and ~290 importing translation units. The single most important result is an **attachment model** under which textual inclusion and `import` of the same entity coexist legally — which is the only model under which moc-generated code can coexist with modules at all. We also document the toolchain traps of Homebrew clang on macOS, a diagnosis method for “impossible” BMI-merge errors, and the analysis/rewriting tooling that converted 179 consumer TUs in an afternoon.

Contents

1	Context and goals	2
2	Toolchain baseline	2
2.1	Non-negotiable CMake settings	2
2.2	<code>import std</code> on Homebrew LLVM / macOS	2
3	The dual-build pattern	3
4	The attachment model — the one decision that matters	4
5	Qt / moc coexistence	4
6	The playbook	5
7	War stories and the diagnosis method	6
7.1	The BMI-merge “mystery” (srcctl.messaging park)	6
7.2	The classic-seam link failure (MessageQueue)	6
7.3	Order-sensitive frontend crashes	7
8	The migration tooling	7
8.1	<code>convertible.py</code> — the frontier analyzer / converter	7
8.2	<code>repro-gmf-attachment/</code> — the conserved reproducer	8
8.3	Draft generators	8
9	The verification protocol	8
10	State at time of writing, and what remains	8

1 Context and goals

The migration target is deliberately conservative:

- **Dual build.** One source tree, two build modes. `SOURCETRAIL_CXX_MODULES=OFF` (default) is the classic textual build and the compatibility guarantee; `ON` builds the same headers into named modules. Every change must keep both modes green (full test suite + a fixed-fixture index-equivalence check).
- **Deep adoption.** Real module structure — partitions, per-layer modules (`srcctl.utility`, `srcctl.file`, `srcctl.data`, `srcctl.storage`, `srcctl.settings`, `srcctl.messaging`, `srcctl.view`, `srcctl.cxx`, ...) — not a single re-export shim.
- **Qt stays Qt.** `moc`, `AUTOMOC`, `Q_OBJECT`, and the Qt headers are used unchanged. Qt itself ships no modules; the coexistence burden is entirely on our side.

Everything below is ordered by importance to someone attempting the same thing, not by the order we discovered it. The war stories (Section 7) explain **why** the rules are what they are.

2 Toolchain baseline

Compiler	Homebrew LLVM/Clang 22.1.8, <code>-std=c++23</code> , <code>libc++</code>
Build	CMake ≥ 4.4 (<code>FILE_SET CXX_MODULES</code> , <code>CMP0155 NEW</code>), Ninja
Platform	macOS (arm64); Apple SDK headers interact with <code>libc++</code> (see Section 2.2)
Qt	Qt 6 via Homebrew frameworks, <code>AUTOMOC ON</code>

2.1 Non-negotiable CMake settings

```
# stdexec (and similar preprocessor-heavy libraries) break clang-scan-deps.
# CMP0155 would scan EVERY C++20+ source in a module-linked target, so scanning
# is off project-wide and enabled per importing file only.
set(CMAKE_CXX_SCAN_FOR_MODULES OFF)

# Per target, per importing TU (see section on per-source keying):
set_source_files_properties(${_importing_tus} PROPERTIES CXX_SCAN_FOR_MODULES ON)
set_property(SOURCE ${_importing_tus} APPEND PROPERTY
    COMPILE_DEFINITIONS SRCTRL_MODULE_BUILD)

# clang 22 defaults to reduced BMIs; full BMIs avoid a class of merge
# regressions (LLVM #207581 manifested under the reduced-BMI default) and a
# CodeGen crash with predeclared operator new/delete (LLVM #166068 area).
target_compile_options(<module-target> PRIVATE -fno-modules-reduced-bmi)
```

Trap. CMake caches module-related compiler detection (import-std support, the `libc++.modules.json` path) at **compiler-detection time**. After changing the toolchain file or the experimental gate, delete `<build>/CMakeFiles/<cmake-version>/` or the change silently does not take.

Trap. `ninja | tail` reports the exit status of `tail`. Capture `ninja`'s own status (`ninja > log 2>&1; echo $?`) — we shipped one “green” build that wasn’t.

2.2 `import std` on Homebrew LLVM / macOS

`import std` works (`SOURCETRAIL_CXX_IMPORT_STD=ON`, implying the modules flag), but required two fixes on this exact toolchain:

1. **CMake wiring:** the experimental gate `UUID` must be set before `project()`, and `CMAKE_CXX_STDLIB_MODULES_JSON` must point at Homebrew’s `<prefix>/lib/c++/libc++.modules.json` (clang’s `-print-file-name` does not find it). Use `/usr/bin/ar` (mach-o), not `llvm-ar`, for the std module archive.

2. A **libc++/SDK header bug**: the stock `std.cppm` fails with `INFINITY / NAN` undeclared in `<complex>`. Root cause: once `<cfloat>` precedes `<cmath> / <complex>`, libc++'s `float.h` wrapper guard (`_LIBCPP_FLOAT_H`) swallows the Apple SDK `math.h` re-include that provides `__need_infinity_nan`. Reproduces in a plain C++23 TU — not modules-specific. Workaround (toolchain file): copy `std.cppm / std.compat.cppm` into the build dir, insert `#include <math.h>` immediately after `module;`, write a patched `libc++.modules.json` next to them, and FORCE-repoint `CMAKE_CXX_STDLIB_MODULES_JSON`.

Two source-level consequences: `import std` exports `std::size_t` but **not** bare `::size_t` (qualify it), and POSIX/C-platform headers (`<time.h>` for `localtime_r`) are not covered by `import std` — they stay textual and unguarded in the wrapper's global module fragment (GMF).

3 The dual-build pattern

`module;` and `export module` must be literal first tokens of a TU — they can never sit behind `#ifdef`. Therefore the dual build is a **header + wrapper split** (the pattern used by {fmt}):

- The **header** keeps all declarations, annotated with `SRCTRL_EXPORT`, and guards includes of **other modules'** headers with `#ifndef SRCTRL_MODULE_PURVIEW`. All definitions are `inline`, in an `.inl` file included at the end of the header (dual-mode ODR merging relies on inline emission from both worlds).
- The **wrapper** (`.cppm`) is module-build-only:

```
module;
// GMF: std/Qt/third-party textual includes, plus classic first-party headers
// whose include closure is module-free (see the GMF closure law).
#include <string>
#include "GroupType.h"

export module srctrl.view;

import srctrl.utility; // imports of other first-party modules
import srctrl.data;

#define SRCTRL_MODULE_PURVIEW
extern "C++" {           // the purview-wide linkage block -- see attachment model
#include "logging.h"     // macro headers self-strip their backend in purview
#include "GraphViewStyle.h"
}
```

- The **scaffolding** (`src/scaffolding/SrctrlModule.h`, deliberately not include-guarded so it re-evaluates per inclusion):

```
#undef SRCTRL_EXPORT
#ifdef SRCTRL_MODULE_PURVIEW
    #define SRCTRL_EXPORT export extern "C++"
#else
    #define SRCTRL_EXPORT
#endif
```

Three preprocessor symbols carry the whole scheme:

<code>SRCTRL_MODULE_PURVIEW</code>	Defined by a wrapper immediately before including its headers. Gates <code>SRCTRL_EXPORT</code> and strips cross-module / std includes from headers.
<code>SRCTRL_MODULE_BUILD</code>	Defined per source file by CMake for each converted consumer TU. Means “this TU imports”. Never define it target-wide (Section 5).
<code>SRCTRL_LOGGING_VIA_IMPORT</code>	Historic per-TU opt-in that strips the logging backend from <code>logging.h</code> so it arrives via <code>import srctrl.logging</code> . Post-pivot it is unnecessary

and **unsafe** whenever the TU's remaining textual closure expands `LOG_*` during the includes — new conversions keep logging fully classic.

4 The attachment model — the one decision that matters

This is the heart of the guide. We ran the migration for eight batches with `SRCTRL_EXPORT = export` (strong module ownership) and paid for it repeatedly; the terminal cost was Qt. The durable model, which we now use everywhere:

Law. Every dual-build entity is attached to the **global module**: `SRCTRL_EXPORT` is `export extern "C++"`, and each wrapper encloses its whole textual purview in an `extern "C++" { }` block. A declaration inside a linkage-specification is attached to the global module (`[module.unit]/7`) while `export` still controls its visibility to importers.

Why this is forced, not merely convenient:

1. **The standard.** Two declarations of one entity attached to different modules are ill-formed when either reaches the other (`[basic.link]/10`); where neither reaches the other you silently get **two distinct entities** with different mangled names. Under bare `export`, any textual parse of a modularized header anywhere in an importing TU's closure is a latent bomb. Under global-module attachment, a textual parse and an import declare the **same** entity and merge (in the include-before-import direction — which the import-placement rule already guarantees).
2. **Mangling.** Clang implements strong ownership: module-attached entities get module-qualified symbols, and module attachment is part of a **class type's** mangling, so it leaks into every signature the type appears in. A classic (non-module) `.cpp` can then never provide a definition importers can link (`MessageQueue::pushMessage(shared_ptr<MessageBase@srcctrl.messaging>)` — undefined, forever). With global-module attachment everything is ordinary-mangled and classic out-of-line definitions link for importer callers too.
3. **moc.** Generated moc TUs can never contain `import`, and `Q_OBJECT` headers need module-owned types (`FilePath`, `MessageListener<T>` bases) **at parse time**, before any import could supply them. The only alternative — include-**after**-import — is documented as unsupported by clang (LLVM #61465).

What you give up: strong-ownership symbol isolation (we never needed it) and the ODR-checking benefits of module linkage. What you keep: BMI-cached compilation, explicit import graphs, `export`-controlled visibility.

Fine print discovered while landing the pivot:

- An out-of-class static member definition cannot itself be `export`ed (not a namespace-scope name) **and** clang attaches a naked purview definition to the module — so it needs a plain (non-export) `extern "C++"` wrap. The purview-wide block makes this automatic; only definitions **outside** the block need care.
- `export` inside `extern "C++"` is fine; nested `extern "C++"` is fine; **nested `export` is ill-formed** — headers included inside an `export extern "C++" { }` block must not use `SRCTRL_EXPORT` themselves.
- Free-function definitions in `.inl`s were the first casualty of a partial pivot (declaration global-module, definition naked-purview → module-attached → “declaration follows declaration in the global module”). Wrapping the whole purview fixed every such case in one stroke, in 36 wrappers, by script.

5 Qt / moc coexistence

With the attachment model in place, moc support reduces to one keying decision:

Law. `SRCTRL_MODULE_BUILD` is defined **per source file** (`CMake set_property(SOURCE ... APPEND PROPERTY COMPILE_DEFINITIONS ...)`, next to the per-file `CXX_SCAN_FOR_MODULES ON`). It means exactly “this TU imports the module graph”. moc-generated TUs and unconverted TUs never see it and keep the classic textual view of every header.

The result, per translation unit of an AUTOMOC'd target:

	Converted widget TU	moc TU (mocs_compilation.cpp)
Defines <code>SRCTRL_MODULE_BUILD</code>	yes (per-source)	no
<code>Q_OBJECT</code> header	textual, classic	textual, classic
Modularized non-Qt headers	guarded out, arrive via <code>import</code>	textual
Entities seen	global-module (merged textual + imported)	global-module (textual)
Mangling	ordinary	ordinary

Both columns name the same entities with the same symbols — that is the whole trick, and it only holds under the attachment model of Section 4.

One Qt-specific footgun survives:

Trap. A converted TU whose **remaining textual closure** expands `LOG_*` macros during the includes (Qt view headers frequently pull a header whose `.inl` logs) cannot use the logging-via-import define: the backend must exist at include time. Keep `logging.h` fully classic in such TUs — post-pivot the textual backend and the module’s are the same entities, so nothing is lost.

`Q_DECLARE_METATYPE` is a macro and cannot cross an import; such invocations are guarded out of purviews (the metatype registration happens in classic TUs).

6 The playbook

The complete rule set as it stands after the pivot (rules that the pivot retired are kept, struck through in spirit, because their **reasons** still explain build failures you may see mid-migration):

1. **Imports after all textual includes.** A textual libcpp header parsed after BMI-merged std declarations trips cannot add `'abi_tag'` in a redeclaration. Include-before-import is also the direction clang supports for mixing textual and imported declarations.
2. **Import what you use — no re-exports between first-party modules.** Corollary, and the single most common manual fix in mass conversion: **visibility does not flow through imports**. When a guarded include used to supply types from other modules transitively, the consumer must name the owner (`import srcctrl.data;`) itself.
3. **Macros never travel through imports.** Macro headers (logging) stay textual; their backends may be modularized behind them.
4. **stdexec in the closure ⇒ unconvertible.** clang-scan-deps chokes on its preprocessor usage, and importer TUs must be scannable. These TUs stay classic (which the dual build makes free).
5. **Dropped includes expose transitive dependencies.** Guarding a header out removes everything it textually supplied — std headers, `<stdcompat/optional>`, classic platform headers. Add them explicitly.
6. **(Retired by the pivot) the mangling law:** under bare `export`, free functions in module content are module-mangled (must be inline), members of attached classes keep ordinary mangling for classic callers only, and importer references are module-mangled. Post-pivot everything is ordinary-mangled; definitions stay inline anyway for dual-mode ODR merging.
7. **Anything a wrapper itself references must be inline in-module** — including vtables of concrete types instantiated in the purview.
8. **Family-internal includes and forward declarations stay unguarded** (same module either way); family forward declarations carry `SRCTRL_EXPORT` (an exported definition after an unexported first declaration is ill-formed).
9. **A wrapper GMF may only include headers whose whole closure is module-free.** Purview guards do not protect a GMF parse — `SRCTRL_MODULE_PURVIEW` is only defined after `export module`. When a classic GMF dependency later joins a module, every wrapper pre-loading it must switch to importing the owner (this was the `srcctrl.messaging` park, Section 7). Enforced by `convertible.py --audit-gmf`.

10. **Cycle-prone aggregates** (a factory naming its whole family) live in a dedicated `.inl` included only by the wrapper (last) plus a classic emission TU whose **non-inline** function odr-uses them — an unused namespace-scope address-take is dropped without emitting anything.
11. **Classic-forever implementation seams** (a `stdexec pimpl`, a platform `impl`): the class and every type in its member signatures must be global-module. Under the pivot this is automatic; the historic explicit form is a purview-gated `export extern "C++" { class MessageQueue {...}; }`.
12. **Only export-bearing headers are import-substitutable**. Classic headers (`Id.h`, `types.h`) are parsed inside purviews too, but their unexported global-module entities are **invisible to importers** — consumers must keep including them textually.

7 War stories and the diagnosis method

7.1 The BMI-merge “mystery” (`srcctl.messaging park`)

The single most instructive failure. Symptom: building a module unit (`srcctl_cxx-frontend.cppm`) failed with

```
error: declaration 'trim' attached to named module 'srcctl.utility:string'
       cannot be attached to other modules
note: utilityString.h:51: also found
```

— with **provably zero** textual parses of `utilityString.h` in the failing TU (`-E / -H clean`), surviving BMI-clean rebuilds, and persisting even with the newly added module wrapper removed from the build. It looked exactly like a compiler bug. It wasn't.

Facts that make such errors diagnosable:

- The diagnostic (`err_multiple_decl_in_different_modules`) is emitted from exactly one place in clang: the **BMI deserialization merger** (`ASTReaderDecl.cpp`), never the parser. The colliding declaration always comes from some **other** BMI in the import closure that textually parsed the header during its own compilation.
- The diagnosis is **order-dependent**: if the global-module copy deserializes first, clang emits only `-wdecls-in-multiple-modules` — which is **off by default**; the other order is a hard, unsuppressable error. That is why reshaping the import graph (or merely sweeping unrelated headers) makes the error appear and vanish.

The method that found it in minutes, after hours of wrong turns:

1. Extract the exact failing command from the ninja log (`FAILED:` blocks include the full command line and the `.modmap` path).
2. The `.modmap` lists every BMI in the closure. BMIs record the paths of all textually parsed inputs — a plain `strings` scan exposes the carrier:

```
grep -o '=[^"]*\.\pcm' failing.modmap | while read p; do
  echo "${strings -a "${p#}" | grep -c 'utilityString\.'} $p"
done | sort -rn # any nonzero count outside the owner is your culprit
```

1. The carrier was `srcctl.cxx:tooling`, whose GMF pre-loaded `MessageStatus.h` from its classic days. When `MessageStatus` joined `srcctl.messaging`, its header reached `utilityString.h` through a purview guard that is **inactive in a GMF** — global-module attachment vs `import srcctl.utility`, ill-formed per `[basic.link]/10`. Clang was right; it was just cruelly quiet about it.

A six-file minimal reproducer (with the two equally-ill-formed-but-accepted contrast shapes that prove the laziness) is conserved at `tools/modules-migration/repro-gmf-attachment/` with a survey of the related LLVM issue history in its README.

7.2 The classic-seam link failure (`MessageQueue`)

Fixing the attachment error exposed the next stratum: the wrapper's instantiated `Message<T>::dispatch()` bodies referenced `MessageQueue@srcctl.messaging::pushMessage(...)`, defined nowhere — the implementation is a classic-forever `stdexec pimpl` TU with ordinary mangling. Wrapping the class in `export extern "C++"` fixed `MessageQueue` itself but **not** the signatures: module attachment is part of the parameter types' mangling, so

`MessageBase`, `MessageFilter`, `MessageListenerBase` all had to become global-module too. This chain — seam class, then every type in its signatures — is what generalized into the full attachment pivot.

7.3 Order-sensitive frontend crashes

One import order (`import srctrl.settings; import srctrl.file;`) produced a clang frontend SEGV (exit 138) in one TU; swapping the two imports fixed it. Alphabetical import order happens to be safe in this codebase. If you hit an inexplicable frontend crash in an importing TU, permute the imports before digging deeper.

8 The migration tooling

Everything lives in `tools/modules-migration/`. The philosophy: the **analyzer** is rerun constantly and must be trustworthy; the **rewriter** is a convenience whose output the dual build arbitrates.

8.1 `convertible.py` — the frontier analyzer / converter

Model. A classic TU is convertible to an importer when its first-party includes are each either (a) a modularized header — replaced by an import of the owning module — or (b) a classic header whose textual closure is clean. Post-pivot, “clean” reduces to: no `stdexec` in the closure (scannability is the only hard blocker left).

How it derives ground truth (nothing is hand-maintained):

- **Module surface** (`module_surface()`): walks every wrapper `.cppm`, splits at the literal `#define SRCTRL_MODULE_PURVIEW` line (splitting at the bare string once misattributed a header because the macro was mentioned in a comment), and maps each purview-included header basename to the wrapper’s export module name — but **only if the header actually contains** `SRCTRL_EXPORT` (unexported classic headers are invisible to importers and must stay textual).
- **Module import graph:** export module / import lines of all wrappers, giving the transitive set of BMIs any import loads (`loaded_closure`) — used to distinguish a genuinely clashing textual parse from harmless include-only coexistence.
- **Closure dirt** (`closure_dirt()`): a transitive walk over the textual include graph collecting (reason, owning-module) pairs — modularized headers, forward declarations of attached types, `stdexec` markers. Header text is pre-stripped of `#ifndef SRCTRL_MODULE_BUILD` regions (guarded content cannot clash by construction) and path-includes resolve first-party only by full suffix match (`<clang/Basic/SourceLocation.h>` must not collide with a first-party `SourceLocation.h` by basename).

Modes, in the order a migration uses them:

```
# 1. Where is the frontier? (default: analyze every .cpp under src/)
tools/modules-migration/convertible.py
#   -> CONVERTIBLE <tu> + exact import block, BLOCKED <tu> + reasons,
#       ranked blocker list (which header gates how many TUs)

# 2. Preprocessed truth instead of the textual scan: use the self-index.
#   (We index our own tree; EDGE_INCLUDE edges are the include graph after
#   preprocessing -- resolves gated headers and exotic spellings exactly.)
tools/modules-migration/convertible.py --index Sourcetrail.srctrl.db

# 3. Convert. Rewrites each TU in place: wraps modularized includes in
#   #ifndef SRCTRL_MODULE_BUILD, appends the import block after the
#   CONTIGUOUS preamble (never after a late #endif -- learned the hard way),
#   prints the CMake entries to add.
tools/modules-migration/convertible.py --apply src/lib_gui/**/*.*.cpp

# 4. Audit wrapper GMFs for the attachment bomb of section 7.1.
tools/modules-migration/convertible.py --audit-gmf # exit 1 on findings
```

How the mass conversion was actually run (batches 9–10, 179 TUs):

```
# per target: list -> apply -> regenerate the CMake importing list from grep
files=$(python3 tools/modules-migration/convertible.py $(find src/lib_gui -name '*.cpp') \
    | grep '^CONVERTIBLE' | awk '{print $2}' \
    | while read n; do find src/lib_gui -name "$n"; done)
echo "$files" | xargs python3 tools/modules-migration/convertible.py --apply
# CMake list = grep truth:
grep -rLE '^import (srcctrl|aidkit)' src/lib_gui --include='*.cpp' | sort
```

(Note the `xargs` : in zsh an unquoted `$files` is a **single** word — two of our “mystery” tool failures were this shell footgun, not the tool.)

Known residual gaps (kept deliberately; the OFF build is the arbiter): `--apply` derives imports from **direct** modularized includes only, so transitively supplied types from other modules surface as compile errors to fix by adding the owner’s import (rule 2’s corollary); generator sweeps do not understand multi-line declarations or `template` + `SRCTRL_EXPORT` ordering.

8.2 repro-gmf-attachment/ — the conserved reproducer

Six files + `run.sh` reproducing the section-7.1 diagnostic exactly, including the two contrast cases clang silently accepts. Point `CXX=` at any clang to re-test a toolchain. Read its README before trusting any “cannot be attached to other modules” error at face value.

8.3 Draft generators

`cpp2inl.py` and `hdr2mod.py` draft the header/.inl split and wrapper skeleton for a new family. They are starting points; their known regex gaps (multi-line signatures, `SRCTRL_EXPORT public:` artifacts, missed `inline` on `= default` members) are listed in the tools README and were all caught by the dual build.

9 The verification protocol

Every slice, no exceptions — the protocol caught every regression this migration ever introduced:

1. **ON:** configure `-DSOURCETRAIL_CXX_MODULES=ON`, full build (capture ninja’s own exit code), full test suite, headless index of a fixed fixture project and compare exact node/edge/error counts (Sourcetrail index `--full <project>.srcctrl.toml`, then `sqlite3` the resulting DB).
2. **OFF:** reconfigure, full build, full test suite. OFF is the default and the compatibility guarantee; it must never be left dirty.
3. **Self-index** (per milestone): index the module-built tree with itself; the module nodes and import edges in the resulting DB are both a dogfood test and the input to `--index` mode.

10 State at time of writing, and what remains

- ~15 first-party modules / 44 module units; ~290 importing TUs across `lib_core`, `lib_gui` (104, most AUTOMOC’d), `lib_cxx`, test, app, indexer.
- The consumer frontier is **closed**: 4 TUs stay classic because `stdexec` makes them unscannable (`TaskBuildIndex.h` consumers, `Schedulers.h`), and 5 are deliberate keepers (dual-switch tests and intentional textual coverage of a module’s headers).
- **Incremental-rebuild benchmark** (M1, debug config, converged-ninja protocol — always verify a second `ninja` reports “no work to do” before and after each touch, or residual work silently leaks into the next measurement):

touch	OFF	ON	ratio
<code>FilePath.h</code> (hot header)	273 TUs / 129 s	326 TUs / 215 s	~1.7×
<code>Edge.cpp</code> (leaf)	1 TU / 5 s	1 TU / 7 s	parity

The dual build makes hot-header edits **dearer** in ON mode by construction: classic TUs still rebuild textually AND the BMI chain plus importers rebuild on top, serialized by BMI dependencies. The ratio improves as consumers

convert ($\sim 1.8\times$ before the mass conversion, $\sim 1.7\times$ after), but the real win requires narrowing the classic textual path itself. Leaf edits are at parity — the module graph costs nothing when it is not touched.

- Remaining planned work: periodic self-index refreshes, and a scanner strategy for stdexec TUs if clang-scan-deps ever learns to cope.

References. Standard: [module.unit]/7, [basic.link]/10–11, [module.interface]/6 (eel.is/c++draft). Clang: “Standard C++ Modules” documentation (clang.llvm.org/docs/StandardCPlusPlusModules.html), LLVM issues #61465 (include-after-import), #61360 / #129525 (cross-module merge defects), #207581 (GMF duplicate merging, fixed on trunk 2026-07), #126373 (friend attachment false positive). Pattern background: ChuanqiXu9, “C++20 Modules: Best Practices” (export extern “C++” / export-using styles). In-tree: context/DESIGN_INDEXER_MODULARIZATION.md (the full phase log), tools/modules-migration/README.md (the living playbook), tools/modules-migration/repro-gmf-attachment/.